

This is a copy of a conversation between ChatGPT & Anonymous.
Report conversation

このコードを、長さを長くせずに簡潔にリファクタリングしてみてください。

```
const duplicatedHotkeys = computed(() => {
  return filterDuplicatedHotkeysWithSetKeyAndName(
    lastRecord.value,
    lastAction.value as HotkeyActionNameType
  );
});

const filterDuplicatedHotkeysWithSetKeyAndName = (
  key: HotkeyCombination,
  action: HotkeyActionNameType
): HotkeySettingType[] => {
  // ホットキー設定ダイアログで何も設定していない時
  if (key === "") return [];
  const results: HotkeySettingType[] = [];
  // argumentKeyの配列を取得。undefinedを代入したら[
  undefined ]が返ってくる
  const argumentKeys =
    getArgumentKeyCombination(getArgumentKey(action));
  for (const argumentKey of argumentKeys) {
    // 探すキー名
    const searchKeyCombination = HotkeyCombination(
      key + (argumentKey === undefined ? "" : " " +
argumentKey)
    );
    // 重複するキーを探す
    const result: HotkeySettingType | undefined =
findDuplicatedHotkeyWithKey(
      searchKeyCombination,
      action
    );
    // 重複するキーがあれば追加する
    if (result !== undefined) {
      results.push(result);
    }
  }
}
```

```

    }
  }
  return results;
};
const findDuplicatedHotkeyWithKey = (
  key: HotkeyCombination,
  action: HotkeyActionNameType
): HotkeySettingType | undefined => {
  return hotkeySettings.value.find(
    (item) =>
      (getArgumentKey(item.action) == undefined
        ? item.combination == key
        : (
            getArgumentKeyCombination(
              getArgumentKey(item.action)
            ) as HotkeyCombination[]
          ).find(
            (argumentKey: HotkeyCombination) =>
              item.combination + " " + argumentKey == key
          )) && item.action != action
      );
};

```

回答は日本語でお願いします。

このコードを簡潔にリファクタリングするには、関数内の処理をより明確にし、不要な比較や代入を省略することが重要です。また、TypeScriptの機能を活用して、コードをより短く、読みやすくすることができます。以下にリファクタリングしたコードを示します。

typescript

```

const duplicatedHotkeys = computed(() =>
  filterDuplicatedHotkeysWithSetKeyAndName(
    lastRecord.value,
    lastAction.value as HotkeyActionNameType
  )
);

const filterDuplicatedHotkeysWithSetKeyAndName = (
  key: HotkeyCombination,

```

```
    action: HotkeyActionNameType
  ): HotkeySettingType[] => {
```

ChatGPT 5.2 ▾

```
.map((argumentKey) =>
  findDuplicatedHotkeyWithKey(
    `${key}${argumentKey ? ` ${argumentKey}` : ""}`,
    action
  )
)
.filter((result): result is HotkeySettingType => result !== undefined)
};
```

```
const findDuplicatedHotkeyWithKey = (
  key: HotkeyCombination,
  action: HotkeyActionNameType
): HotkeySettingType | undefined =>
  hotkeySettings.value.find((item) => {
    const itemKeyCombination = getArgumentKey(item.action)
      ? `${item.combination} ${
        getArgumentKeyCombination(getArgumentKey(item.action)) as HotkeyCombination
      }`
    : item.combination;
    return itemKeyCombination === key && item.action !== action;
  });
```

このリファクタリングでは以下の変更を行いました：

- 三項演算子の使用**： `argumentKey == undefined ? "" : " " + argumentKey` を `${argumentKey ? `${argumentKey}` : ""}` に変更して、コードをより簡潔にしました。
- フィルタリングの簡略化**：結果が `undefined` でないものだけをフィルタリングすることで、最後に結果の配列を返す処理を簡潔化しました。この際に、TypeScriptのタイプガード（`result is HotkeySettingType => result !== undefined`）を利用して、明確に結果が `HotkeySettingType` 型の要素のみを配列に含めるようにしました。
- 短絡評価の利用**：不要な空文字列のチェック（`if (key == "") return [];`）を、`!key` で簡潔に行いました。

- 4. テンプレートリテラルの使用：**文字列の結合を、テンプレートリテラルを用いてより読みやすく表現しました。

これらの変更により、元のコードの意図を保ちつつ、より簡潔で読みやすいコードにリファクタリングすることができました。